

Final Project: Report

Souren Prakash, Kevin Martone

DS 4420, April 16

Github: https://github.com/kevinmartone/DS4420_FinalProject

Introduction and Literature Review

The biological kingdom Fungi presents a rich and underexplored domain for applied machine learning research. This project focuses on optimizing mushroom cultivation for hobbyists and home growers, a practice that has grown significantly in popularity alongside rising consumer interest in medicinal and culinary fungi. Species such as reishi, lion's mane, and king oyster mushrooms have seen surging demand due to their health benefits, including improved cognitive function, sustained energy, and general immune support. As commercial products incorporating these fungi proliferate, home cultivation has emerged as an accessible and rewarding alternative for individuals seeking to grow their own supply.

Understanding the biology of fungi is essential for this work. What we recognize as a mushroom is merely the fruiting body. The reproductive structure of the organism is analogous to the fruit of a tree. The true body of the fungus is the mycelium, a network of thread-like hyphae that colonizes a nutrient-rich substrate. Successfully cultivating mushrooms depends on maintaining a clean colonization environment. Mycelium is slow-growing and highly vulnerable to contamination from competing molds, bacteria, and other microbial organisms. If a contaminant establishes itself within the substrate before the mycelium can fully colonize, it will often outcompete and ultimately destroy the culture.

A persistent challenge for hobbyists is identifying contamination visually. Many common contaminants, particularly in early stages, can be difficult to distinguish from

healthy mycelial growth without prior experience. Misidentification leads to either the unnecessary discarding of viable cultures or, more critically, the continued use of a contaminated substrate. While current research does exist for contamination detection via machine learning methods, these approaches tend to require more costly or specialized technology. Further, where image-based methods do exist, they are designed for controlled agar plate environments rather than general hobbyist images. This project addresses that gap by developing a classification model to determine whether a substrate is contaminated, using two model types: a Convolutional Neural Network (CNN) and a Bayesian machine learning model.

Ujir et al. (2024) proposed two complementary approaches for automated mycelium classification: an HSV histogram-based entropy method and a CNN architecture (MConNet) built on MobileNetV3 transfer learning. Evaluated on 1,400 images of *Pleurotus ostreatus* cultures in standardized petri dishes, their CNN method achieved 97.81% accuracy for contamination detection. However, their setup relies on controlled, top-down imaging of petri dishes. These conditions do not reflect the varied substrates and imaging environments encountered in home cultivation. Furthermore, their classification is binary (contaminated vs. non-contaminated), whereas identifying the type of contaminant would provide more actionable guidance for growers.

Similarly, Liu and Li (2022) demonstrate another use case of machine learning for fungi classification, focusing on identifying different growth stages as they pertain to assisting the growth of host plants. This study employs techniques such as MLP and SVM, but relies on data collected via SWIR (short-wave infrared) spectroscopy, a technology that is not accessible to the general hobbyist. Our project aims to extend these frameworks by targeting contamination classification on uncontrolled hobbyist images, making detection accessible without specialized equipment.

Methodology

For this project, we focused on two types of models: a Convolutional Neural Network and a Bayesian model. For both, we ensured that the number of images was balanced between the two classes, contaminated and non-contaminated, to avoid class imbalance issues during training.

Data Collection + Pre-processing

We searched the internet for a dataset of hobbyist culture images. This proved difficult, as there is no standardized public dataset for this use case. More structured datasets, such as those using agar plate images, do exist online, but these make contamination easier to identify due to the controlled imaging environment. They don't reflect the messier, more variable conditions of real hobbyist cultivation. While the lack of standardization in our dataset introduces challenges, it is also precisely what makes this problem interesting and practically relevant. The final dataset we landed from is sourced from Kaggle and, as mentioned above, came with an even split. Two hundred fifty clean and two hundred fifty contaminated images. A majority of the images are taken of the mycelium within its growth environment without too much background image pollution.

Once our dataset was found, the first preprocessing step involved ensuring uniform image dimensions. For both the CNN and Bayesian approaches, all images were resized to 128×128 pixels using the PIL library in Python. The resized images were fed directly into the CNN with no further preprocessing. Greyscale conversion was unnecessary as RGB colour values are a key identifier of contamination.

The Bayesian model required additional preprocessing since numerical features are necessary. Our approach involved four steps:

1. Convert images to a dataframe
2. Normalise all pixel values to [0,1]
3. Reduce from $3 * 128^2 = 49152$ columns to 24, 8 bins per colour channel

4. Normalise bin counts within each colour channel

The dataframe is constructed by flattening each image's RGB values into a 1D array, producing one row per image with columns following the pattern `r_0`, `g_0`, `b_0`, `r_1`, `g_1`... With 49,152 features, MCMC sampling required one independent chain per feature per class, making the runtime infeasible. To address this, pixel values were binned into 8 ranges per channel (0–0.125, 0.125–0.25, and so on), and the number of pixels in each bin was counted. These counts were then normalised within each channel by dividing by the total pixel count for that channel, keeping all values in [0,1] and reducing the feature space to 24 columns.

Convolutional Neural Network

We developed two variants of CNN during this project. The first was a custom architecture built using the TensorFlow Keras library. This model used a 3×3 kernel size across all convolutional layers, with ReLU as the primary activation function and a sigmoid output layer for binary classification. Max pooling with a 2×2 kernel was applied throughout. Initial performance was poor, and experimenting with leaky ReLU provided only marginal improvement.

After researching CNN architectures suited to our specific problem, we found that pretrained models designed to capture color and texture features would be a strong fit, given that visual contamination detection depends heavily on those cues. We settled on ResNet50 as our transfer learning backbone, and performance improved substantially. For both CNN variations, we performed stratified k-fold cross-validation. Given the relatively small size of our dataset, roughly 50 test images per class before cross-validation, and the high degree of within-class variability in hobbyist images, this was the most reliable way to produce a model with any generalizability.

Bayesian Model

As normalised values fall in $(0,1)$, a Beta prior is placed on μ and a Gamma prior on τ (precision = $1/\text{variation}$). These priors are non-conjugate with the Gaussian likelihood, meaning no closed-form posterior exists and MCMC sampling is required. The basis for our Metropolis Hastings approach was sourced from [BayesRulesBook](#). Metropolis-Hastings is run independently per feature per class (48 chains total):

- 2000 iterations, 500 burn-in discarded, thinning by 5
- Initial attempts used burn-in of 200 and thinning of 2, resulting in higher autocorrelation between samples and poorer chain mixing
- Proposal step size for μ was reduced from 0.05 to 0.01. After normalisation, feature values are concentrated in a narrow range, and a smaller step size better matches the posterior width

For making the predictions, the log predictive likelihood is computed with Monte Carlo integration. Rather than plugging in single-point estimates for μ and τ , the Gaussian likelihood is averaged across all posterior samples per feature. Log predictive likelihoods are summed across all 24 features and combined with the log class prior. The class with the higher combined score is assigned via argmax. This approach more closely mirrors that of a Naive Bayes approach. We went with this approach as the small dataset would face a significant overfitting risk with a logistic weights-based model. Additionally, both logistic and naive Bayes approaches rely on independence between features. Neither of which would really be met, seeing where the bins are being generated from, but a naive Bayes approach seemed the lesser of two evils. Doing so also removes variables from the sampling approach, further reducing run-time. These decisions were influenced by a paper inspecting the differences in performance between logistic regression and Naive Bayes for landslide susceptibility(Tsangaratos). Overall, the paper found Naive Bayes to outperform the logistic model.

Results

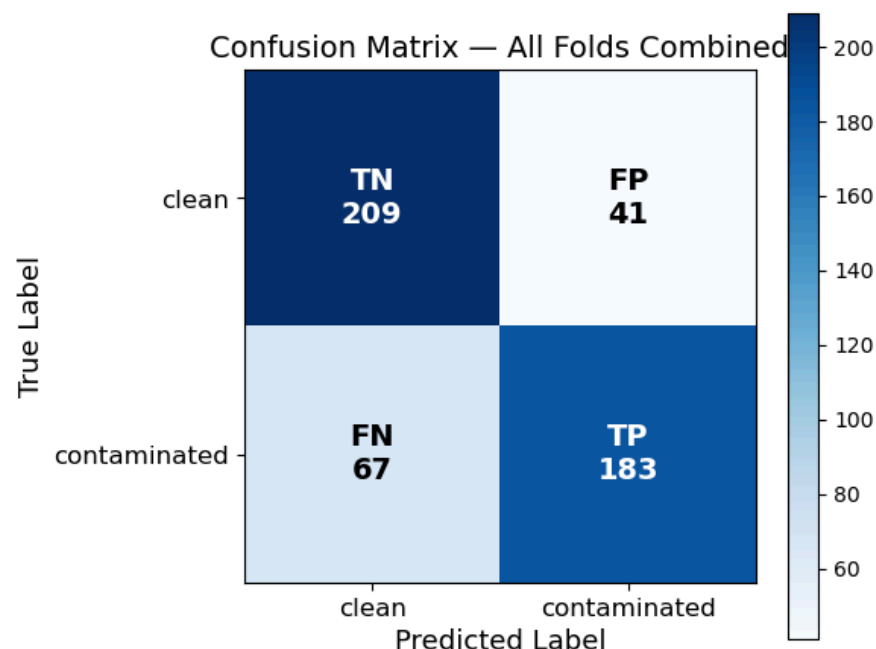
Convolutional Neural Network - Keras

The initial Keras model that we created did not have high accuracy. After running it a couple of times and trying to change the activation functions and kernel sizes, we pivoted to using ResNet to better detect the color and textures associated with contamination of mycelium. Below is the loss and accuracy for the Keras model. While better than the Bayesian results you will see below, we believed further improvement should be possible.

Test Loss: 0.5079, Test Accuracy: 0.6346

Convolutional Neural Network - ResNet50

In the confusion matrix of our predictions, the main issue is that false negatives are the highest error in our model. This is an issue as it directly contradicts our base idea that you should throw away a contaminated sample. If the model classifies more false positives, this is better: a person can decide not to discard, but for the model to say a sample is clean when it is contaminated is worse.



CROSS-VALIDATION SUMMARY

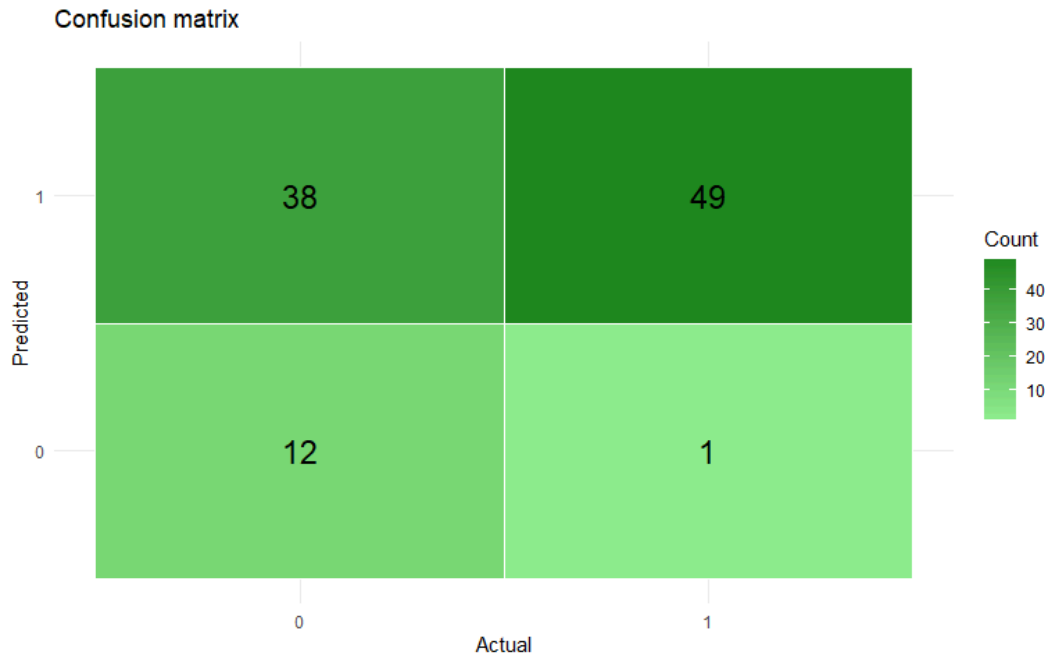
clean F1 – mean: 0.791 std: 0.067
contaminated F1 – mean: 0.770 std: 0.068
Accuracy – mean: 0.784 std: 0.057

	precision	recall	f1-score	support
clean	0.80	0.86	0.83	50
contaminated	0.85	0.78	0.81	50
accuracy			0.82	100

Bayesian

Unlike the previous model, the Bayesian results are much less prone to predicting clean when the image was not in fact clean. You can see in the confusion matrix below that there was only one case out of 100 in which clean was falsely predicted. Meaning our probability of truth given we predict clean is roughly $1/13 = 7.69\%$. Compared to the ResNet50 results, which give you a probability of $41/250 = 16.4\%$. Meaning we are twice as likely to be wrong about a clean prediction from the ResNet50 model compared to the Bayesian model. One thing to note is that the validation used differs between models(K-fold vs standard 80/20 train-test split).

That is where the Bayesian advantages end. Overall accuracy and other metrics go to the ResNet50 model. The Bayesian model is just significantly more likely to predict contaminated overall, resulting in a lower accuracy score of 61% compared to the 78.4% for ResNet50.



Interpretation

The reason why the ResNet 50 did a better job overall is that the architecture is more complicated and does a better job at detecting color and texture. These are the most important features for humans to classify something as contaminated or not. Generally speaking, it would have been better to have somewhat standardized data. The objective of this project is to enable hobbyists or smaller-scale growers to have a check for their examples, so we do want it to apply to a variety of pictures. However, without data that effectively shows the different types of contaminations and what they look like on mycelium, it is difficult to create the correct contaminated predictions. With an accuracy of 78.4% during the cross-validation, this model currently works best as a first check but not an end-all be-all.

Given the use case of small farmers or independent groups, we really don't want prospective users to be keeping mycelium that has been contaminated due to an incorrect prediction. As above, we would still recommend a first run through with the ResNet50 to gather contaminated and clean predictions. Then, to verify these predictions, we would recommend a user to take the would-be clean images and feed

those through the Bayesian model. Doing so is able to leverage the strengths of both models and may improve results overall. This is dependent on the fact that the Bayesian models accuracy maintains after performing some cross-fold validation to ensure the accuracy of the model to a larger test set.

Conclusion

The models developed in this project show real promise as a preliminary check or automated screening tool for mushroom growers who are unsure about the health of their mycelium. At this stage, they are not yet reliable enough to serve as a grower's sole method of assessment, given the variability in hobbyist images and the limited size of our training data. That said, the combined approach, utilizing the models consecutively, demonstrates a possible use case and is our recommended method. If these models continue to improve, the application could extend well beyond hobbyist use. At a larger scale, an automated contamination detection system could be integrated into commercial or small-group farming operations to flag potential issues at key points in the colonization process, reducing manual inspection overhead and minimizing crop loss. However, periodic checks by experienced professionals would always be recommended, as these models are trained on past cases and may lack some of the breadth of possible contaminations a true expert understands.

Future Work

The most pressing next step is gathering more standardized image data. The within-class variability in our current dataset is the single biggest constraint on model performance, and expanding the dataset with more consistent imaging conditions would have a large impact on results. Alongside that, we plan to continue uploading images from at-home cultivation to build a more realistic and representative dataset over time.

On the modeling side, implementing cross-fold validation for the Bayesian model, which we applied to the CNN but not yet to the Bayesian component, would allow for a

fairer comparison between the two approaches. Long-term, the most impactful extension would be shifting from binary to multiclass classification. Either creating or sourcing a dataset with specific contamination labels (e.g., green mold, cobweb mold, bacterial blotch) would allow growers to not just detect contamination but identify its cause, making the tool genuinely actionable rather than just diagnostic.

Appendix

CNN Keras:

```
# %%  
# establishing model architecture  
model = Sequential([  
  
    # For these elements we could change the kernal size? or the type of  
    function?  
    # additionally maybe the size of the pooling?  
    Conv2D(32, (3, 3), activation='relu', input_shape=(256, 256, 3)),  
    MaxPooling2D((2, 2)),  
  
    Conv2D(64, (3, 3), activation='relu'),  
    MaxPooling2D((2, 2)),  
  
    Conv2D(128, (3, 3), activation='relu'),  
    MaxPooling2D((2, 2)),  
  
    Conv2D(256, (3, 3), activation='relu'),  
    MaxPooling2D((2, 2)),  
  
    # Perceptron part  
  
    Flatten(),  
    Dense(256, activation='relu'),  
    # we would want to change this to multi classification for the  
    differnt types of contamination  
    Dense(1, activation='sigmoid')  
])  
  
# %%
```

```

# reading that adam is better for this problem than SGD
# trying that out

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss= 'binary_crossentropy',
    metrics=['accuracy'])

#TRAINING
# %%
history = model.fit(train_ds, validation_data=val_ds, epochs=10)

# %%
loss, acc = model.evaluate(test_ds)
print(f"Test Loss: {loss:.4f}, Test Accuracy: {acc:.4f}")

```

CNN ResNet50 :

```

def build_model():
    K.clear_session()

    base = ResNet50(
        input_shape=(256, 256, 3),
        include_top=False,
        weights='imagenet'
    )
    base.trainable = False

    inputs = tf.keras.Input(shape=(256, 256, 3))

```

```

x      = base(inputs, training=False)
x      = tf.keras.layers.GlobalAveragePooling2D()(x)
x      = Dense(128, activation='relu')(x)
x      = Dropout(0.5)(x)
outputs = Dense(1, activation='sigmoid')(x)

model = tf.keras.Model(inputs, outputs)
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
return model

# 4. STRATIFIED K-FOLD CROSS VALIDATION
#%%
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

fold_results = []
all_y_true   = []
all_y_pred   = []
best_acc     = -1
best_model   = None
best_test_ds = None

for fold, (train_val_idx, test_idx) in enumerate(skf.split(all_paths,
all_labels)):
    print(f"\n{'='*40}")
    print(f"Fold {fold + 1}/5")
    print(f"{'='*40}")

    val_size = int(0.15 * len(train_val_idx))
    train_idx = train_val_idx[val_size:]
    val_idx   = train_val_idx[:val_size]

```

```

train_ds = make_dataset(all_paths[train_idx], all_labels[train_idx],
shuffle=True)
val_ds   = make_dataset(all_paths[val_idx],   all_labels[val_idx])
test_ds  = make_dataset(all_paths[test_idx],  all_labels[test_idx])

print(f"Train: {len(train_idx)} | Val: {len(val_idx)} | Test:
{len(test_idx)}")

model = build_model()

early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=3, restore_best_weights=True
)

model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=[early_stop],
    verbose=1
)

y_true, y_pred = [], []
for images, labels in test_ds:
    preds = model.predict(images, verbose=0)
    y_pred.extend((preds[:, 0] >= 0.5).astype(int))
    y_true.extend(labels.numpy())

all_y_true.extend(y_true)
all_y_pred.extend(y_pred)

report = skm.classification_report(y_true, y_pred,
target_names=class_names, output_dict=True)
cm      = skm.confusion_matrix(y_true, y_pred)
fold_results.append(report)

```

```

    print(skm.classification_report(y_true, y_pred,
target_names=class_names))
    print("Confusion Matrix:")
    print(cm)

    if report['accuracy'] > best_acc:
        best_acc      = report['accuracy']
        best_model     = model
        best_test_ds   = test_ds

```

Bayesian:

```
``{r prior-specification}
```

```
# Use color histogram features (24 cols) not raw pixels
```

```
feat_cols <- setdiff(names(color_features), c("filename", "label"))
```

```
# Pull histogram rows using same split indices from train-test-split
```

```
train_df <- color_features[split_idx, ]
```

```
test_df <- color_features[-split_idx, ]
```

```
class_counts <- table(train_df$label)
```

```
class_priors <- class_counts / sum(class_counts)
```

```
cat("Class priors:\n"); print(class_priors)
```

#Normalizing each color bin separately. Ie if red total count is 130, and one bin has 20 then we get a value of 20/130 for that bin.

```
normalise_hist <- function(df, feature_cols) {
```

```
  df_norm <- df
```

```
  r_cols <- grep("^r_bins", feature_cols, value = TRUE)
```

```
  g_cols <- grep("^g_bins", feature_cols, value = TRUE)
```

```
  b_cols <- grep("^b_bins", feature_cols, value = TRUE)
```

```

for (cols in list(r_cols, g_cols, b_cols)) {
  chunk  <- as.matrix(df[, cols])
  row_sums <- rowSums(chunk)
  row_sums[row_sums == 0] <- 1 #No div by 0
  df_norm[, cols] <- chunk / row_sums
}
df_norm
}

#Post normalization
train_df <- normalise_hist(train_df, feat_cols)
test_df  <- normalise_hist(test_df, feat_cols)

cat("Number of features:", length(feat_cols), "\n")
cat("Normalised feature range: [",
    round(min(as.matrix(train_df[, feat_cols])), 4), ",",
    round(max(as.matrix(train_df[, feat_cols])), 4), "]\n")
#Expect mean roughly centered at .5 via beta, gamma for tau demonstrates we are uncertain
a_mu <- 2; b_mu <- 2
a_tau <- 2; b_tau <- 1
# Using log to avoid multiplication
log_prior <- function(mu, tau) {
  if (mu <= 0 || mu >= 1 || tau <= 0) return(-Inf)
  dbeta(mu, a_mu, b_mu, log = TRUE) +
  dgamma(tau, a_tau, b_tau, log = TRUE)
}
#Gaussian stdev 1/sqrt(tau)
#Mult with log becomes addition
log_likelihood_feat <- function(x, mu, tau) {
  sum(dnorm(x, mean = mu, sd = 1 / sqrt(tau), log = TRUE))
}

log_posterior_feat <- function(x, mu, tau) {
  log_likelihood_feat(x, mu, tau) + log_prior(mu, tau)
}

```



```
...
```

```
```{r model-fitting}
```

```
mh_sampler <- function(x, n_iter = 2000, burnin = 500, thin = 5,
```

```
 prop_sd_mu = 0.01, prop_sd_tau = 0.1) {
```

```
 mu_cur <- mean(x)
```

```
 tau_cur <- 1 / var(x)
```

```
 if (!is.finite(tau_cur)) tau_cur <- 1 #Set to one if var(x) = 0(all vals same)
```

```
 mu_cur <- min(max(mu_cur, 1e-6), 1 - 1e-6) #prevent mu from hitting 1 and 0 for beta issues
```

```
 samples <- matrix(NA, nrow = n_iter, ncol = 2,
```

```
 dimnames = list(NULL, c("mu", "tau")))
```

```
 lp_cur <- log_posterior_feat(x, mu_cur, tau_cur)
```

```
 for (i in seq_len(n_iter)) {
```

```
 mu_prop <- mu_cur + rnorm(1, 0, prop_sd_mu)
```

```
 tau_prop <- tau_cur * exp(rnorm(1, 0, prop_sd_tau))
```

```
 lp_prop <- log_posterior_feat(x, mu_prop, tau_prop)
```

```
 if (is.finite(lp_prop) && is.finite(lp_cur)) {
```

```
 log_accept <- lp_prop - lp_cur + log(tau_prop) - log(tau_cur) #Check if value shows
 improvement ie positive result(won't be finite if nt)
```

```
 if (is.finite(log_accept) && log(runif(1)) < log_accept) {
```

```
 mu_cur <- mu_prop
```

```
 tau_cur <- tau_prop
```

```
 lp_cur <- lp_prop
```

```
 }
```

```
 }#Record regardless accept/reject proposed
```

```
 samples[i,] <- c(mu_cur, tau_cur)
```

```
 }
```

```
 keep_idx <- seq(burnin + 1, n_iter, by = thin)
```

```
 samples[keep_idx,]
```

```
}
```

```
set.seed(42)
```

```
cat("Running MCMC...\n")
```

```
compute_posterior_samples <- function(df_train, feature_cols, class_priors) {
 classes <- names(class_priors)
 all_samps <- list()
 total <- length(feature_cols) * length(classes)
 done <- 0
```

```
 for (cl in classes) {
 subset <- df_train |>
 filter(as.character(label) == as.character(cl)) |> # safe string match
 select(all_of(feature_cols)) |>
 as.matrix()
```

```
 all_samps[[cl]] <- lapply(seq_len(ncol(subset)), function(j) {
 done <-< done + 1
 cat(sprintf("\r Chain %d / %d", done, total))
 flush.console()
 mh_sampler(subset[, j])
 })
 }
 cat("\n")
 all_samps
}
```

```
posterior_samples <- compute_posterior_samples(train_df, feat_cols, class_priors)
#Feature 1 class 0/non-contaminated
samp0 <- posterior_samples[["0"]][[1]]
cat("\nPosterior samples shape:", dim(samp0), "\n")
cat("Posterior mean mu (feature 1, class 0):", round(mean(samp0["mu"]), 4), "\n")
cat("Posterior mean tau (feature 1, class 0):", round(mean(samp0["tau"]), 4), "\n")
#Each loop s becomes new row of values
log_predictive <- function(x_new, samples) {
 log_liks <- apply(samples, 1, function(s) {
```

```

 dnorm(x_new, mean = s["mu"], sd = 1 / sqrt(s["tau"]), log = TRUE)
 })
 max_ll <- max(log_liks)
 max_ll + log(mean(exp(log_liks - max_ll)))
}

predict_one <- function(obs, posterior_samples, class_priors) {
 log_posteriors <- sapply(names(class_priors), function(cl) {
 samps <- posterior_samples[[cl]]
 log_lik <- sum(mapply(log_predictive, obs, samps))
 log(class_priors[cl]) + log_lik
 })
 as.integer(names(which.max(log_posteriors)))
}

#predicts for each row in the DF
predict_bayes <- function(df_new, feature_cols, posterior_samples, class_priors) {
 X <- as.matrix(df_new[, feature_cols])
 apply(X, 1, predict_one,
 posterior_samples = posterior_samples,
 class_priors = class_priors)
}

test_preds <- predict_bayes(test_df, feat_cols, posterior_samples, class_priors)
cat("\nPredictions (first 10):", test_preds[1:10], "\n")
cat("Actual (first 10):", test_df$label[1:10], "\n")
...

```

## Bibliography

- [1] Ujir, H., Hipiny, I., Bolhassan, M. H., & Ku Azir, K. N. F. (2024). (IJACSA) International Journal of Advanced Computer Science and Applications,.  
<https://pdfs.semanticscholar.org/75ff/daceb8726502211d5d6f8aa5588859507931.pdf>
- [2] Liu, Z., & Li, Y. (2022). Fungi Classification in Various Growth Stages Using Shortwave Infrared (SWIR) Spectroscopy and Machine Learning. *Journal of Fungi*, 8(9), 978.  
<https://doi.org/10.3390/jof8090978>
- [3] Johnson, Alicia A., Miles Q. Ott, and Mine Dogucu. "Chapter 7: MCMC under the Hood." *Bayes Rules! An Introduction to Applied Bayesian Modeling*, CRC Press, 2022, [www.bayesrulesbook.com/chapter-7](http://www.bayesrulesbook.com/chapter-7).
- [4] Tsangaratos, Paraskevas, and Ioanna Ilia. "Comparison of a Logistic Regression and Naïve Bayes Classifier in Landslide Susceptibility Assessments: The Influence of Models Complexity and Training Dataset Size." *CATENA*, vol. 145, 2016, pp. 164–179.